

UNIVERSITÉ DE TECHNOLOGIE D'ABIDJAN

Département Informatique — Licence 3

PROJET 7

Application Multi-Cloud

*Render + Railway · Docker · GitHub Actions ·
VMware*

Étudiant	Filière	Année académique
Yavo Jean-Pierre Koné Abdoul Yama Ziega Karl-Loic	Informatique Génie Logiciel Licence 3	2025 / 2026

Docteur MOYA

Application TaskFlow — Todo List multi-cloud avec virtualisation Docker

Table des matières

Table des matières	2
1. Introduction et objectifs du projet	4
1.1 Objectifs	4
1.2 Stack technique	4
2. Développement de l'application Flask	5
2.1 Fonctionnalités de l'application	5
2.2 Structure du projet	5
2.3 Routes de l'application	5
3. Conteneurisation avec Docker	7
3.1 Dockerfile	7
3.2 Commandes Docker utilisées	7
Construction de l'image	7
Test en local	7
Connexion à Docker Hub	7
Tag de l'image	7
Push vers Docker Hub	7
4. Déploiement sur Render	9
4.1 Configuration du service Render	9
4.2 Deploy Hook	9
5. Déploiement sur Railway	11
5.1 Configuration du service Railway	11
5.2 Commandes Railway CLI	11
6. Pipeline CI/CD avec GitHub Actions	13
6.1 Fonctionnement du pipeline	13
6.2 Secrets GitHub configurés	13
6.3 Fichier deploy.yml	14
7. Virtualisation avec VirtualBox	16
7.1 Architecture de virtualisation	16
7.2 Configuration des deux VMs	16
7.3 Installation de Docker sur chaque VM	16
Mise à jour du système	16
Installation de Docker	16
Démarrage et activation automatique	16
Ajout des droits Docker à l'utilisateur	17
Vérification	17
7.4 Déploiement du conteneur sur vm-render	17
Téléchargement de l'image	17

Lancement du conteneur	17
Vérification du conteneur	17
Récupération de l'IP de la VM.....	17
7.5 Déploiement du conteneur sur vm-railway	17
Lancement du conteneur	17
7.6 Commandes de gestion Docker sur les VMs	18
8. Résultats et validation	21
8.1 URLs de déploiement.....	21
8.2 Validation du pipeline CI/CD.....	21
9. Conclusion	22
Compétences acquises	22

1. Introduction

Ce projet a pour objectif de déployer une même application web sur deux plateformes cloud distinctes, en utilisant la conteneurisation Docker et un pipeline CI/CD automatisé via GitHub Actions. L'approche multi-cloud garantit la disponibilité et la résilience de l'application.

1.1 Objectifs

- Développer une application web Flask (Todo List) fonctionnelle
- Conteneuriser l'application avec Docker
- Déployer sur deux clouds distincts : Render et Railway
- Automatiser le déploiement avec GitHub Actions
- Virtualiser l'environnement avec VMware (deux VMs)

1.2 Stack technique

Composant	Technologie	Rôle
Application	Python Flask	Backend web — Todo List
Conteneurisation	Docker	Empaquetage et portabilité
Registre d'images	Docker Hub	Stockage de l'image Docker
Cloud 1	Render	Hébergement plateforme (PaaS)
Cloud 2	Railway	Hébergement plateforme (PaaS)
Virtualisation	VMware + Ubuntu	VMs locales avec Docker
CI/CD	GitHub Actions	Pipeline de déploiement automatisé
Versioning	Git + GitHub	Gestion du code source

2. Développement de l'application Flask

L'application TaskFlow est une Todo List complète développée en Python avec le framework Flask. Elle intègre plusieurs fonctionnalités avancées et affiche dynamiquement la plateforme cloud sur laquelle elle est déployée.

2.1 Fonctionnalités de l'application

- Ajout, suppression et marquage des tâches
- Priorités : Haute / Moyenne / Basse avec codes couleur
- Catégories : Travail, Personnel, Étude, Réunion, Urgent, Autre
- Date d'échéance avec détection automatique des tâches en retard
- Recherche de tâches par mot-clé
- Filtres combinables : statut + catégorie
- Barre de progression globale
- Badge cloud dynamique (affiche Render, Railway ou local)
- Endpoint /health retournant les statistiques en JSON

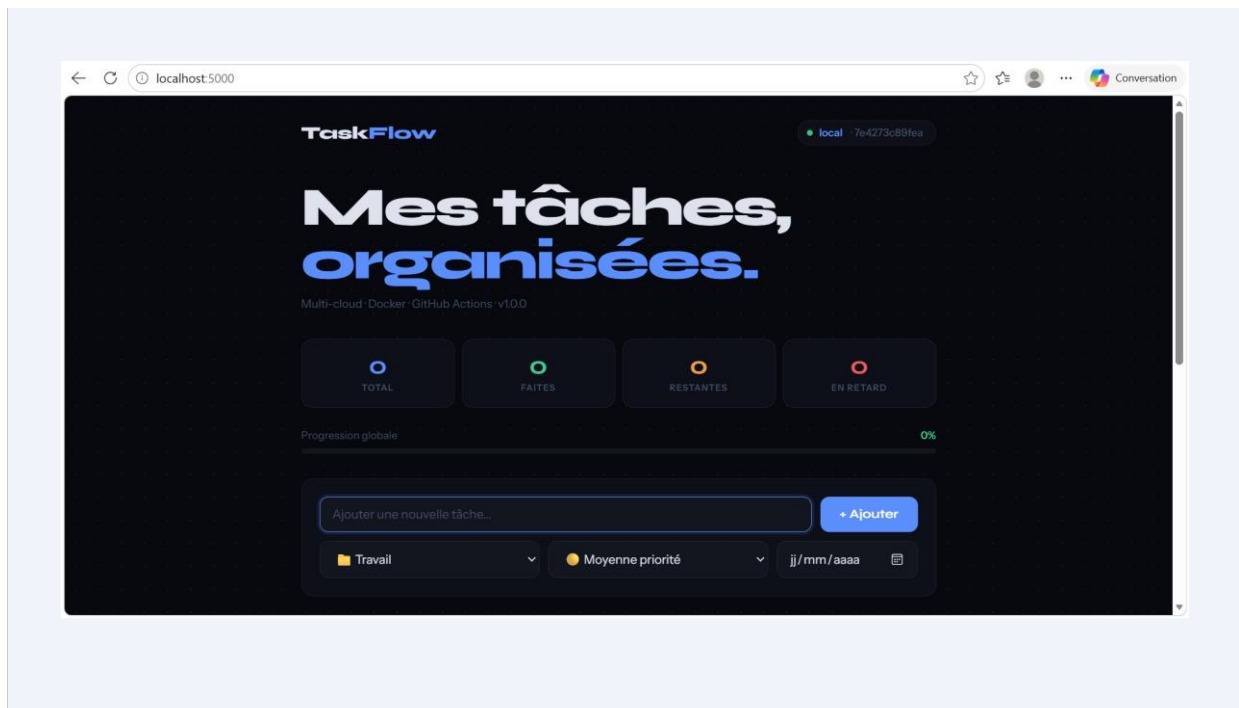
2.2 Structure du projet

```
multicloud-todo-app/  
├── app/  
│   ├── app.py          ← logique Flask  
│   ├── requirements.txt ← dépendances Python  
│   └── templates/  
│       └── index.html   ← interface utilisateur  
├── Dockerfile           ← conteneurisation  
├── docker-compose.yml   ← test en local  
├── .github/  
│   └── workflows/  
│       └── deploy.yml   ← pipeline CI/CD
```

2.3 Routes de l'application

Route	Méthode	Description
/	GET	Page principale — liste des tâches
/add	POST	Ajouter une nouvelle tâche
/toggle/:id	POST	Marquer une tâche comme terminée
/delete/:id	POST	Supprimer une tâche
/health	GET	Vérification de l'état de l'app (JSON)

Capture d'écran — Interface principale de l'application TaskFlow



Capture d'écran — Endpoint /health — réponse JSON



3. Conteneurisation avec Docker

Docker permet d'empaqueter l'application et toutes ses dépendances dans une image portable. La même image sera déployée identiquement sur les deux clouds.

3.1 Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY app/ .
RUN pip install -r requirements.txt
EXPOSE 5000
CMD ["unicorn", "--bind", "0.0.0.0:5000", "app:app"]
```

3.2 Commandes Docker utilisées

Construction de l'image

```
| docker build -t koneabdoul/multicloud-todolist-app-todo-app:latest .
```

Test en local

```
| docker-compose up --build
```

Connexion à Docker Hub

```
| docker login
```

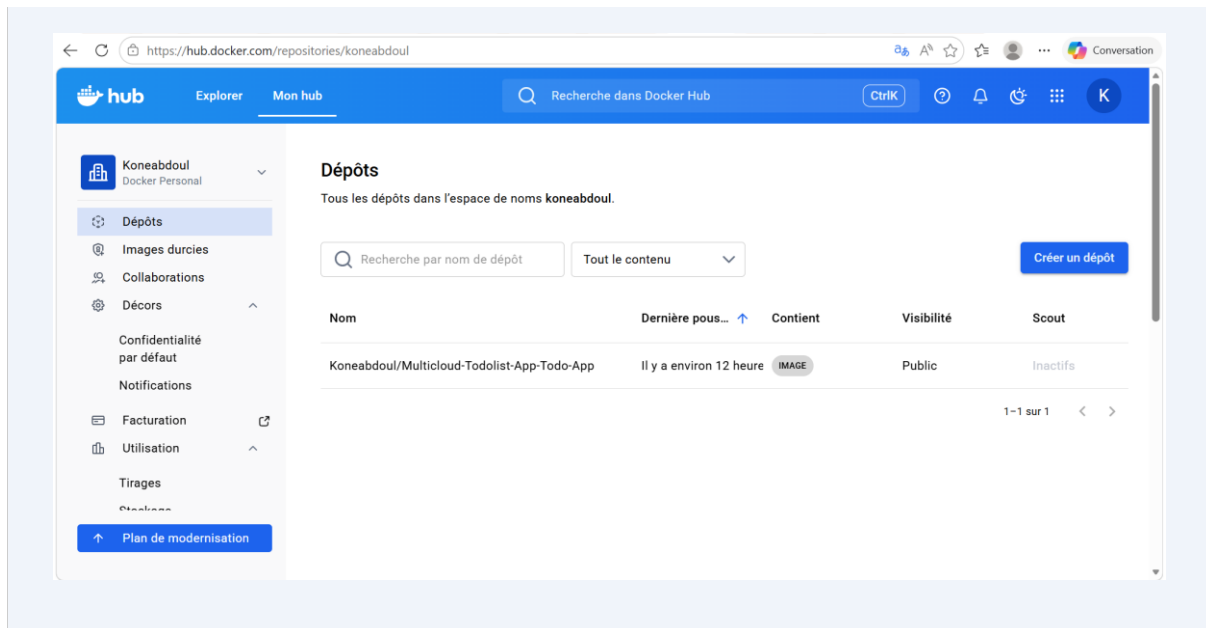
Tag de l'image

```
| docker tag multicloud-todolist-app-todo-app koneabdoul/multicloud-todolist-app-todo-app:latest
```

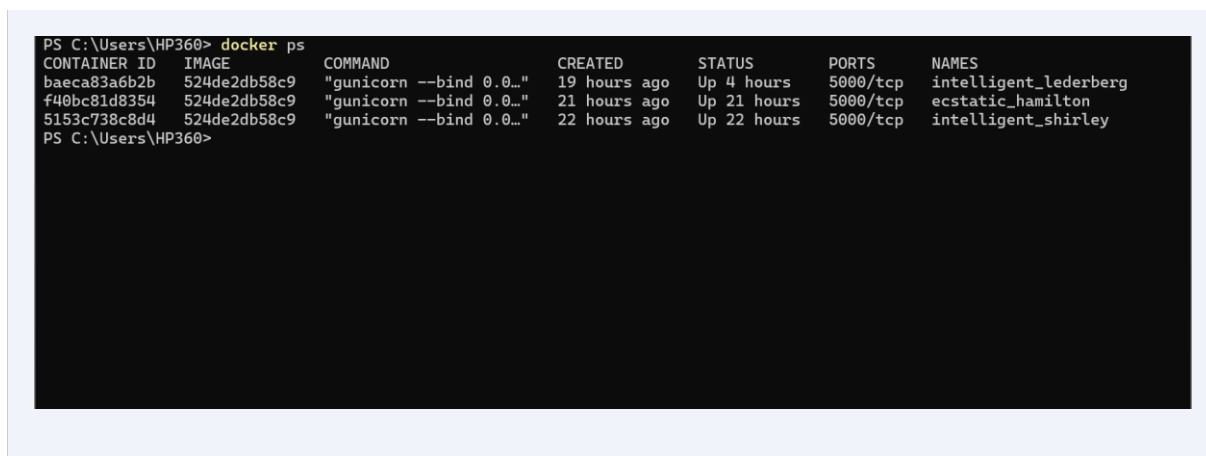
Push vers Docker Hub

```
| docker push koneabdoul/multicloud-todolist-app-todo-app:latest
```

Capture d'écran — Docker Hub — image publiée koneabdoul/multicloud-todolist-app-todo-app



Capture d'écran — docker ps — conteneur en cours d'exécution en local



4. Déploiement sur Render

Render est une plateforme cloud PaaS (Platform as a Service) qui héberge l'application directement depuis une image Docker Hub. Elle gère automatiquement l'infrastructure sous-jacente.

4.1 Configuration du service Render

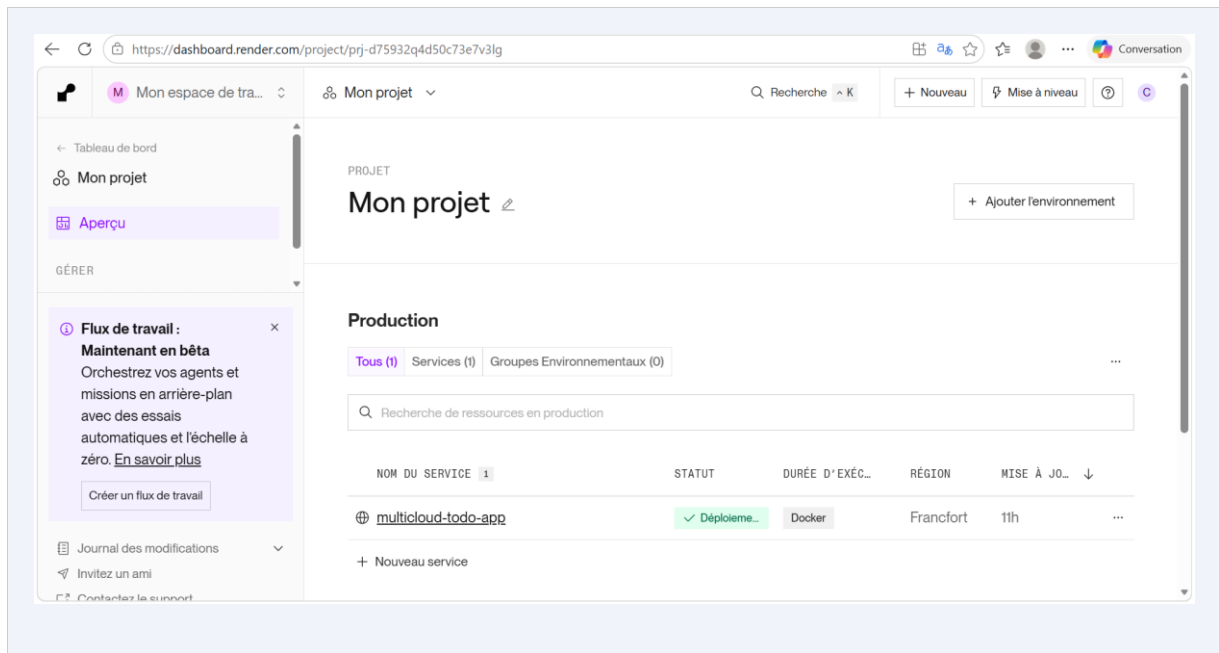
Paramètre	Valeur
Nom du service	multicloud-todo-render
Type	Web Service
Source	Image Docker Hub
Image	koneabdoul/multicloud-todolist-app-todo-app:latest
Région	Frankfurt (EU Central)
Plan	Free
Variable CLOUD_NAME	Render

4.2 Deploy Hook

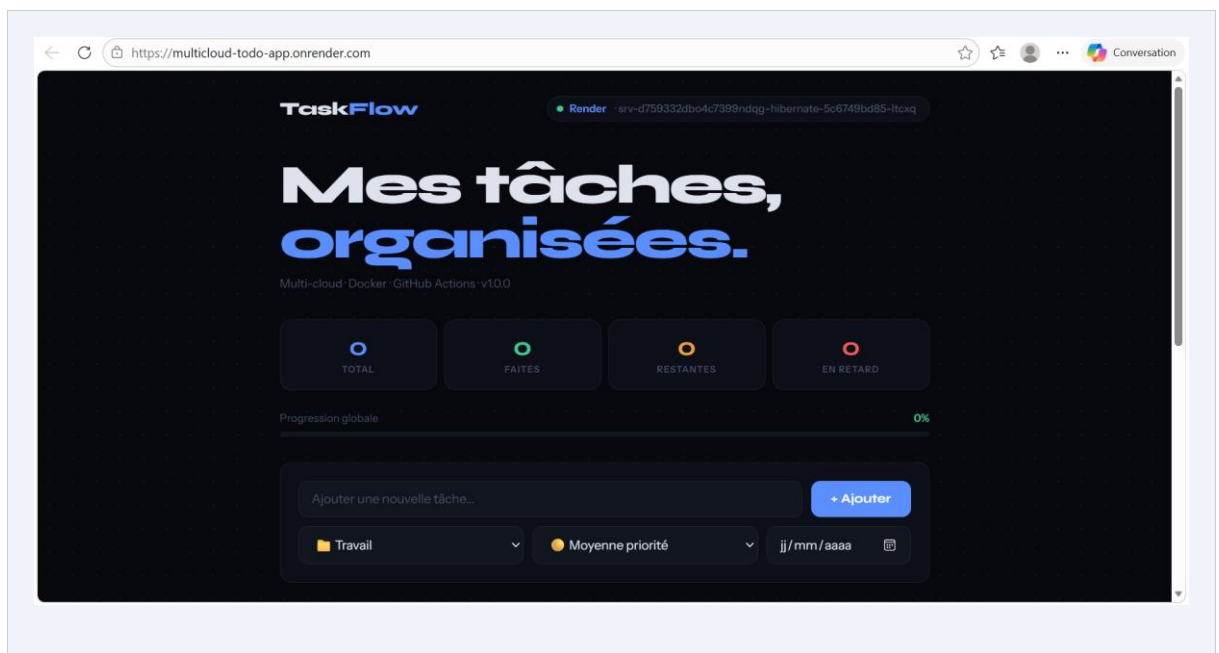
Le Deploy Hook est une URL unique fournie par Render. Lorsque GitHub Actions envoie une requête POST à cette URL, Render redéploie automatiquement l'application avec la dernière image Docker.

```
| curl -X POST "$RENDER_DEPLOY_HOOK"
```

Capture d'écran — Dashboard Render — service déployé et actif



Capture d'écran — Application accessible sur l'URL Render — badge 'Render' visible



5. Déploiement sur Railway

Railway est la seconde plateforme cloud utilisée dans ce projet. Elle complète Render pour constituer l'architecture multi-cloud. Le déploiement s'effectue via la Railway CLI intégrée au pipeline GitHub Actions.

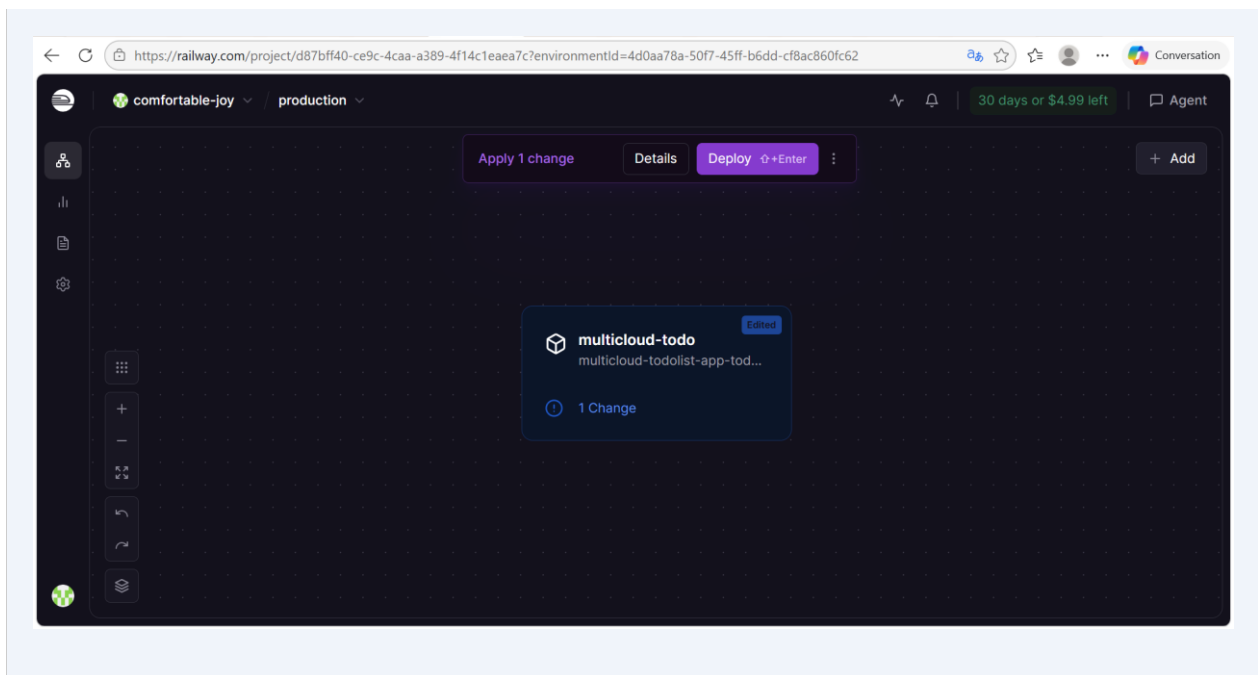
5.1 Configuration du service Railway

Paramètre	Valeur
Nom du service	multicloud-todo
Type	Web Service
Source	Image Docker Hub
Image	koneabdoul/multicloud-todolist-app-todo-app:latest
Variable CLOUD_NAME	Railway
Token	Stocké dans GitHub Secrets

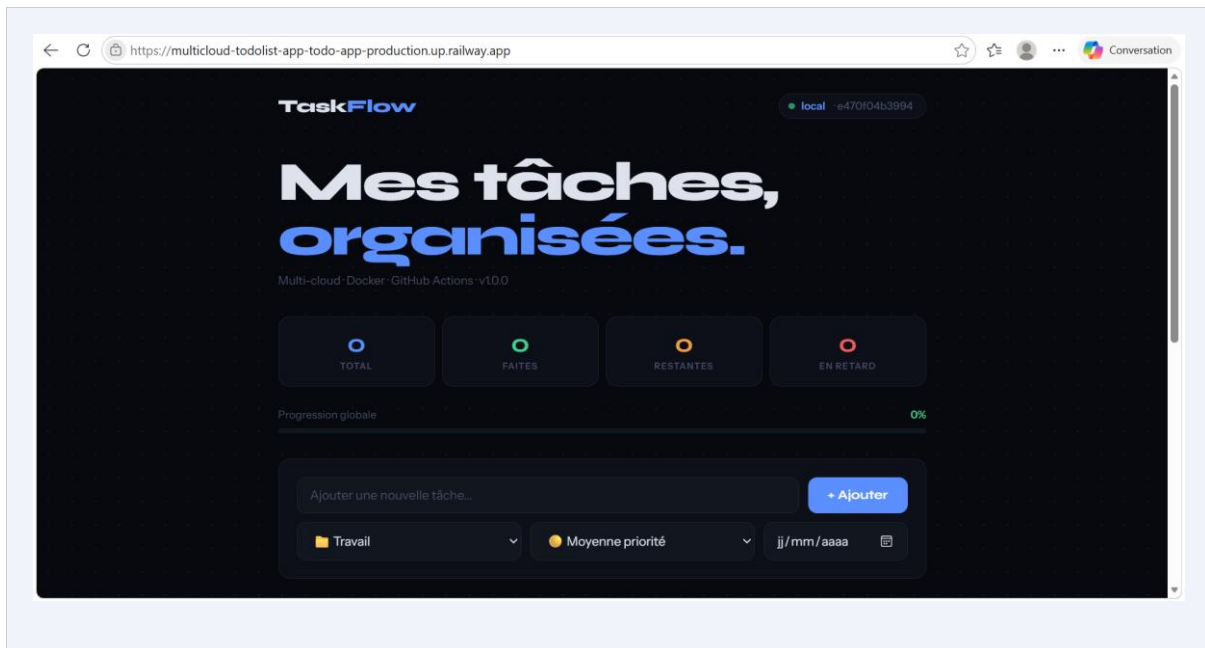
5.2 Commandes Railway CLI

```
npm install -g @railway/cli
railway up --detach --service multicloud-todo
```

Capture d'écran — Dashboard Railway — service déployé et actif



Capture d'écran — Application accessible sur l'URL Railway — badge 'Railway' visible



6. Pipeline CI/CD avec GitHub Actions

GitHub Actions automatise l'intégralité du cycle de déploiement. Dès qu'un commit est poussé sur la branche main, le pipeline se déclenche automatiquement et déploie la nouvelle version sur les deux clouds.

6.1 Fonctionnement du pipeline

Job	Dépendance	Actions réalisées
build-and-push	Aucune (premier)	Checkout → Login Docker Hub → Build image → Push image
deploy-render	build-and-push	Appel du Deploy Hook Render via curl
deploy-railway	build-and-push	Installation Railway CLI → railway up

6.2 Secrets GitHub configurés

Nom du secret	Description
DOCKER_USERNAME	Nom d'utilisateur Docker Hub (koneabdoul)
DOCKER_PASSWORD	Mot de passe Docker Hub
RENDER_DEPLOY_HOOK	URL de déclenchement du déploiement Render
RAILWAY_TOKEN	Token d'authentification Railway CLI

6.3 Fichier deploy.yml

```
name Deploy MultiCloud

# Se déclenche à chaque push sur la branche main
on
  push
    branches
      main

jobs

# -----
# JOB 1 : Construire l'image Docker et la pousser sur Docker Hub
# -----
build-and-push
  name Build & Push Docker Image
  runs-on ubuntu-latest

  steps
    # 1. Récupérer le code du repo
    name Checkout code
    uses actions/checkout@v4

    # 2. Se connecter à Docker Hub
    name Login to Docker Hub
    run |
      echo " secrets DOCKER_PASSWORD " | docker login \
      -u " secrets DOCKER_USERNAME " \
      --password-stdin

    # 3. Construire l'image Docker
    name Build Docker image
    run |
      docker build -t secrets DOCKER_USERNAME /multicloud-todolist-app-
todo-app:latest .

    # 4. Pousser l'image sur Docker Hub
    name Push Docker image
    run |
      docker push secrets DOCKER_USERNAME /multicloud-todolist-app-
todo-app:latest

# JOB 2 : Déclencher le redéploiement sur Render
# (attend que le Job 1 soit terminé)
deploy-render
  name Deploy to Render
```

```
runs-on ubuntu-latest
needs build-and-push

steps
  name Trigger Render Deploy Hook
  run |
    curl -X POST "      secrets RENDER_DEPLOY_HOOK      "
    echo "Render deploy triggered ✔"

# -----
# JOB 3 : Redéployer sur Railway via CLI
# -----
deploy-railway
  name Deploy to Railway
  runs-on ubuntu-latest
  needs build-and-push

  steps
    name Checkout code
    uses actions/checkout@v4

    name Install Railway CLI
    run |
      npm install -g @railway/cli

    name Deploy to Railway
    run |
      railway up --service "multicloud-todo" --detach
  env
    RAILWAY_TOKEN      secrets RAILWAY_TOKEN
```

7. Virtualisation avec VirtualBox

Deux machines virtuelles ont été créées avec VirtualBox pour démontrer la virtualisation complète de l'application. Chaque VM exécute Ubuntu Server avec Docker installé, et héberge un conteneur TaskFlow avec une variable d'environnement différente.

7.1 Architecture de virtualisation

```
graph TD
    A[PC Windows (machine physique)] --> B[VMware (hyperviseur de type 2)]
    B --> C[VM 1 : vm-render (Ubuntu + Docker + conteneur CLOUD_NAME=Render)]
    B --> D[VM 2 : vm-railway (Ubuntu + Docker + conteneur CLOUD_NAME=Railway)]
```

7.2 Configuration des deux VMs

Paramètre	vm-render	vm-railway
Système	Ubuntu Server 22.04	Ubuntu Server 22.04
RAM	2 GB	2 GB
CPU	2 cores	2 cores
Disque	20 GB (VDI)	20 GB (VDI)
Réseau	Accès par pont	Accès par pont
CLOUD_NAME	Render	Railway

7.3 Installation de Docker sur chaque VM

Mise à jour du système

```
sudo apt update && sudo apt upgrade -y
```

Installation de Docker

```
sudo apt install docker.io -y
```

Démarrage et activation automatique

```
sudo systemctl start docker
sudo systemctl enable docker
```


Ajout des droits Docker à l'utilisateur

```
sudo usermod -aG docker $USER  
newgrp docker
```

Vérification

```
docker --version
```

7.4 Déploiement du conteneur sur vm-render

Téléchargement de l'image

```
sudo docker pull koneabdoul/multicloud-todolist-app-todo-app:latest
```

Lancement du conteneur

```
sudo docker run -d \  
  --name taskflow-render \  
  --restart unless-stopped \  
  -p 5000:5000 \  
  -e CLOUD_NAME=Render \  
  -e APP_VERSION=1.0.0 \  
  koneabdoul/multicloud-todolist-app-todo-app:latest
```

Vérification du conteneur

```
docker ps
```

Récupération de l'IP de la VM

```
ip addr show | grep "inet 192"
```

7.5 Déploiement du conteneur sur vm-railway

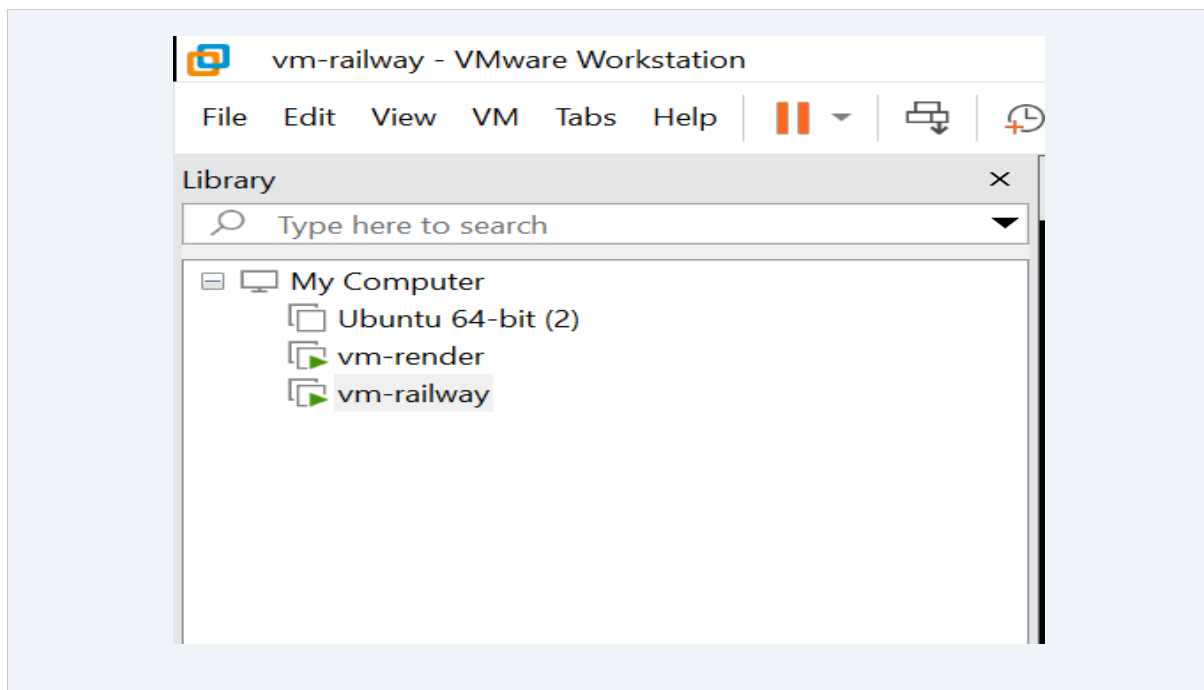
Lancement du conteneur

```
sudo docker run -d \  
  --name taskflow-railway \  
  --restart unless-stopped \  
  -p 5000:5000 \  
  -e CLOUD_NAME=Railway \  
  -e APP_VERSION=1.0.0 \  
  koneabdoul/multicloud-todolist-app-todo-app:latest
```

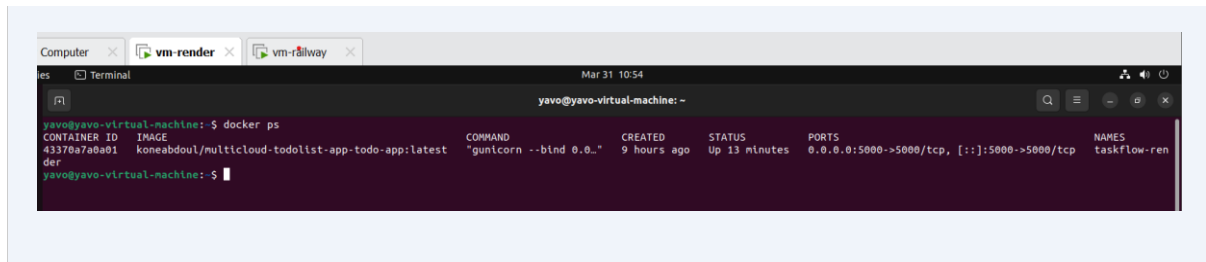
7.6 Commandes de gestion Docker sur les VMs

Commande	Description
<code>docker ps</code>	Lister les conteneurs en cours d'exécution
<code>docker ps -a</code>	Lister tous les conteneurs (actifs et arrêtés)
<code>docker start taskflow-render</code>	Démarrer le conteneur vm-render
<code>docker start taskflow-railway</code>	Démarrer le conteneur vm-railway
<code>docker stop taskflow-render</code>	Arrêter le conteneur
<code>docker restart taskflow-render</code>	Redémarrer le conteneur
<code>docker logs taskflow-render</code>	Voir les logs de l'application
<code>docker logs -f taskflow-render</code>	Suivre les logs en temps réel
<code>docker inspect taskflow-render</code>	Détails complets du conteneur
<code>docker rm taskflow-render</code>	Supprimer le conteneur
<code>docker images</code>	Lister les images téléchargées
<code>docker rmi [image]</code>	Supprimer une image
<code>sudo systemctl restart docker</code>	Redémarrer le service Docker

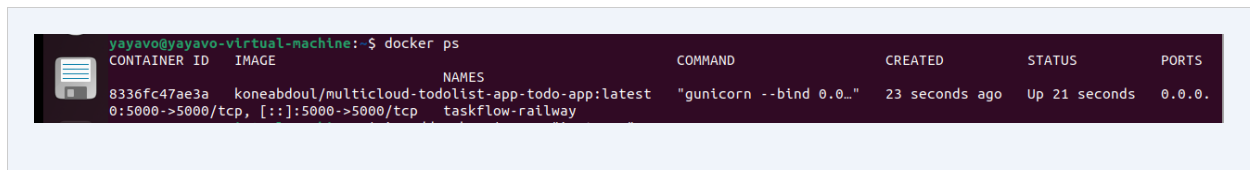
Capture d'écran — VirtualBox — les deux VMs vm-render et vm-railway démarrées



Capture d'écran — Terminal vm-render — docker ps (conteneur taskflow-render actif)



Capture d'écran — Terminal vm-railway — docker ps (conteneur taskflow-railway actif)



Capture d'écran — Navigateur Windows — app sur vm-render (badge 'Render')



Capture d'écran — Navigateur Windows — app sur vm-railway (badge 'Railway')



8. Résultats et validation

8.1 URLs de déploiement

Environnement	URL d'accès	Badge affiché	Statut
Local (Docker Compose)	http://localhost:5000	local	✓ Fonctionnel
VM VirtualBox (Render)	http://192.168.204.130:5000	Render	✓ Fonctionnel
VM VirtualBox (Railway)	http://192.168.204.131:5000	Railway	✓ Fonctionnel
Render (cloud)	https://multicloud-todo-app.onrender.com	Render	✓ Fonctionnel
Railway (cloud)	https://multicloud-todo-app-todo-app-production.up.railway.app	Railway	✓ Fonctionnel

8.2 Validation du pipeline CI/CD

Le pipeline GitHub Actions a été validé avec succès. Chaque push sur la branche main déclenche automatiquement les 3 jobs :

- Job 1 (build-and-push) : construction et publication de l'image Docker
- Job 2 (deploy-render) : redéploiement automatique sur Render
- Job 3 (deploy-railway) : redéploiement automatique sur Railway

9. Conclusion

Ce projet a permis de mettre en pratique les concepts fondamentaux du déploiement multi-cloud moderne. L'application TaskFlow tourne identiquement sur deux plateformes distinctes grâce à la conteneurisation Docker, garantissant la portabilité et la cohérence entre les environnements.

La virtualisation avec VirtualBox illustre concrètement comment Docker s'exécute dans un environnement isolé, simulant le comportement des VMs en production. Le pipeline GitHub Actions démontre l'importance de l'automatisation dans le cycle de développement moderne (CI/CD).

Compétences acquises

- Développement d'une API REST avec Python Flask
- Conteneurisation d'application avec Docker et Docker Compose
- Publication d'images sur Docker Hub
- Déploiement sur des plateformes PaaS (Render, Railway)
- Virtualisation avec VirtualBox et gestion de VMs Ubuntu
- Configuration d'un pipeline CI/CD avec GitHub Actions
- Gestion de secrets et variables d'environnement
- Architecture multi-cloud et haute disponibilité